

Facultad de Ciencias
Escuela de Computación
Aplicaciones con Tecnología Internet
Semestre 1-2026

Laboratorio #5

04/06/2026

Por:
Mauricio González, CI: 30722334

Primero se realizó un análisis del código, en el cual no había ningún uso de la variable `this` (pues sólo eran usadas funciones flechas para ciertas funciones dependientes del contexto), por lo que se agregaron 3 funciones para el uso del `this` y su posterior análisis:

```
// Función normal (window o undefined en modo estricto)
```

```
function metodoThisNormal() {  
  console.log("this:", this);  
}
```

```
// Función en objeto, apunta a objeto
```

```
const objetoPrueba = {  
  nombre: "objeto",  
  metodoThisObjeto: function() {  
    console.log("this de objeto:", this);  
  }  
};
```

```
// función flecha, apunta al contexto en donde este
```

```
const metodoThisFlecha = () => {  
  console.log("this de flecha:", this);  
};
```

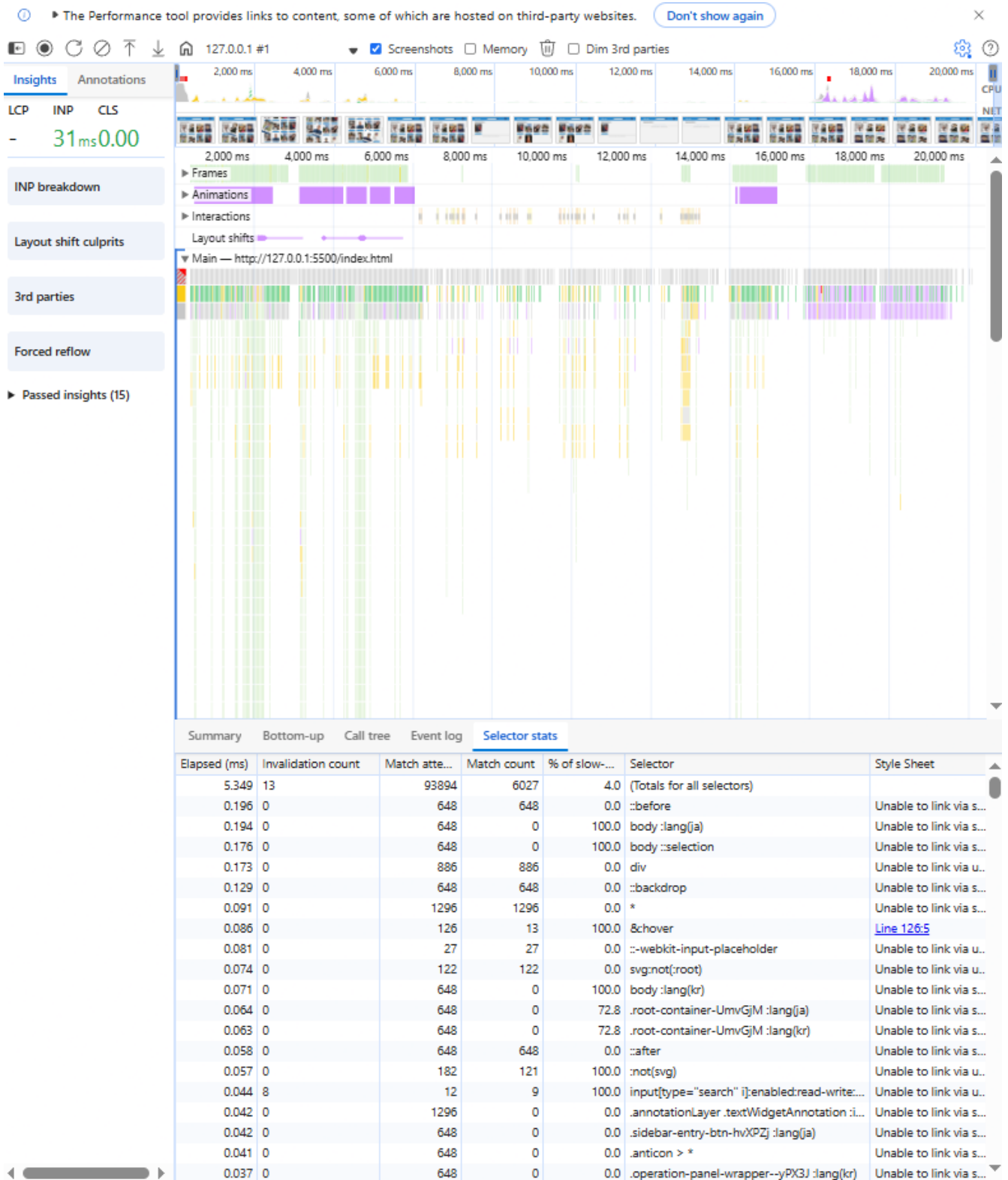
Éste código arrojaba a consola el siguiente resultado:

```
this: ▶ Window http://127.0.0.1:5500/index.html?lang=ES  
this de objeto: ▶ Object { nombre: "objeto", metodoThisObjeto: metodoThisObjeto() ↗ }  
this de flecha: ▶ Window http://127.0.0.1:5500/index.html?lang=ES
```

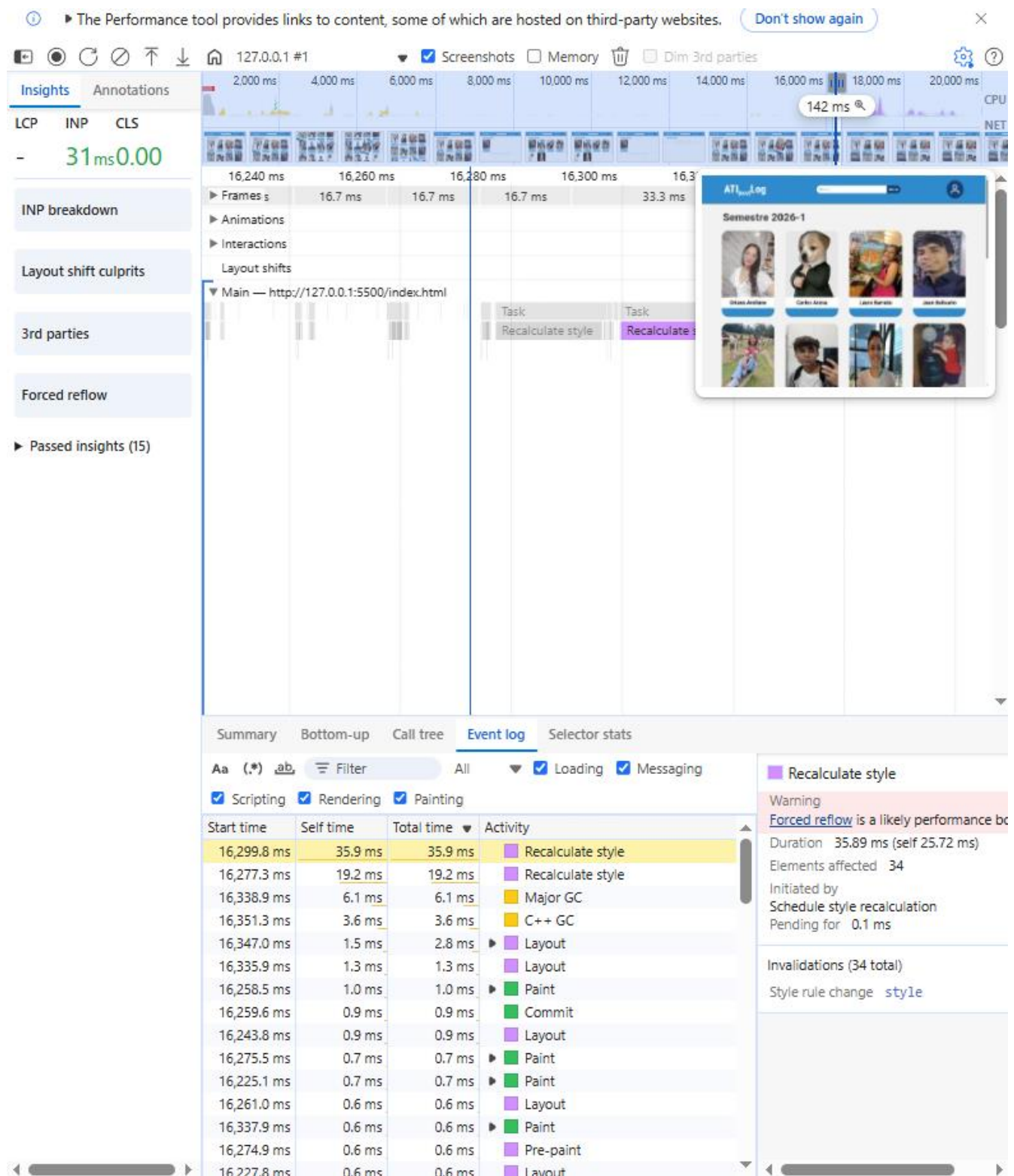
Imagen 1. Resultado de Consolas de las funciones con `this`

Como se puede apreciar, el primer log devuelve como resultado `window`, esto porque es una función ejecutándose en el contexto global, y al no estar en modo estricto devuelve `window`, la segunda función, la del objeto, devuelve una referencia al dueño del objeto, en este caso, el objeto con nombre: “objeto”, y en el último caso, la función flecha devuelve el contexto del contenedor en donde se encuentra, pues no tiene uno propio, siendo éste nuevamente el `window`.

Posteriormente entramos a las DevTools, y activando los selectores CSS verificamos el rendimiento de la página con una captura en donde movíamos los perfiles, hacíamos búsqueda y cambiábamos el tamaño de la página, esto nos devolvió los siguientes resultados:



Imágen 2. Captura del Selector Stats de DevTools



Imágen 3. Captura del Event Log de DevTools

En la Imágen 2. podemos ver cómo el tiempo de recalculado de todos los estilos de 5.349 milisegundos, lo cuál es aceptable para una página (Quizá ésto debiéndose a la simplicidad de nuestra página), aunque, el selector universal y el div tienen altos conteos (1296 y 886 cada uno), lo cual quizá podría ser más costoso si se escalara la página. Sin embargo, actualmente los selectores más costosos

son el ::before, body :lang(ja) y el div, aunque sus tiempos no sobrepasan los 0,2 milisegundos.

Un cambio que sí podíamos realizar (Que fue conversado durante la clase práctica) es aplicar el cambio de **will-change: transform** al selector **.perfil: hover** para indicarle al navegador que anticipe animaciones futuras y así pueda preparar y optimizar los recursos gráficos de un elemento antes de que comience el movimiento.

Posteriormente, en la configuración de la pestaña de Performance en DevTools configuramos el CPU Throttling para que tuviera un cuarto del rendimiento de nuestro procesador, y vimos el rendimiento de la página en esas condiciones:

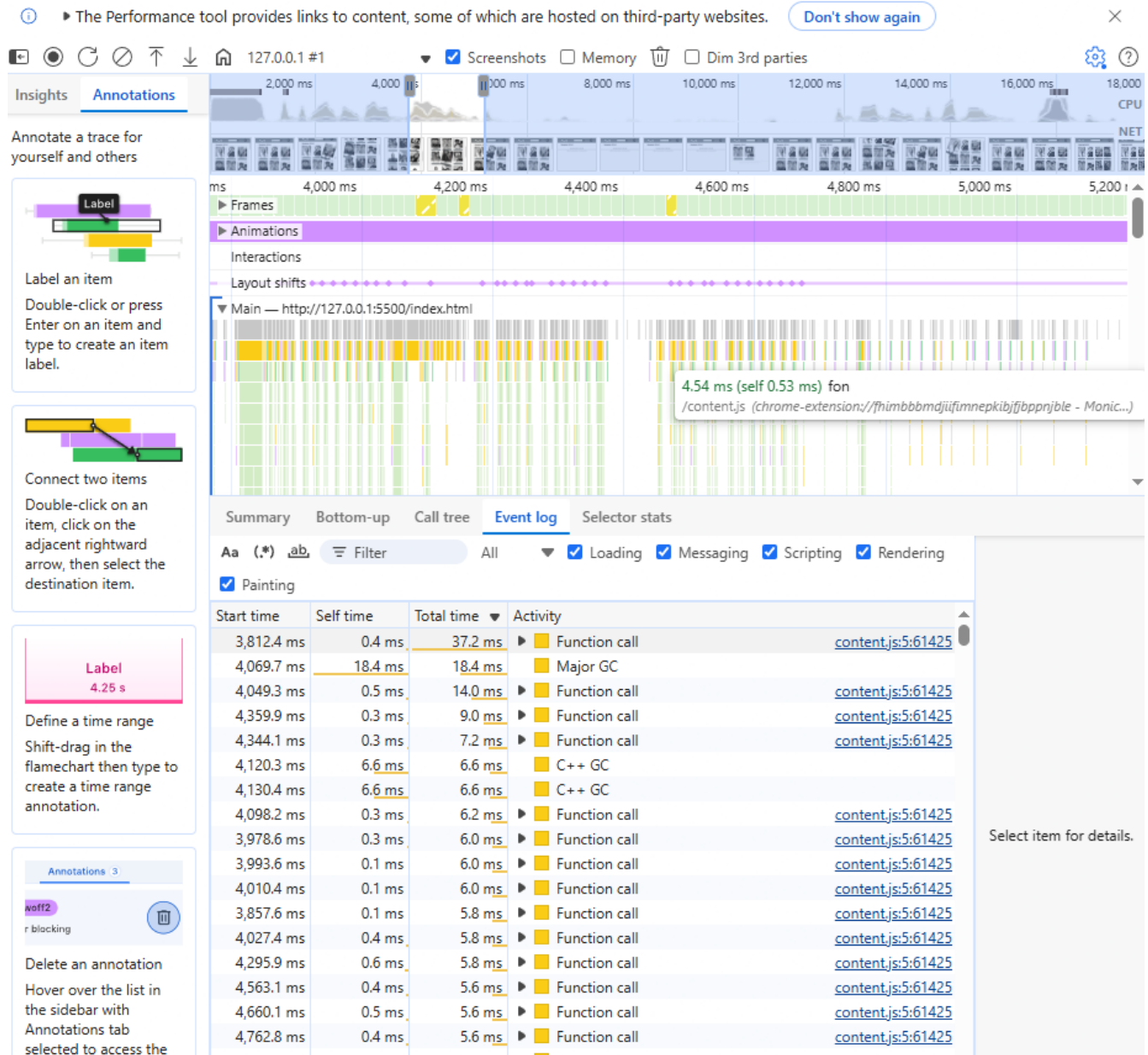
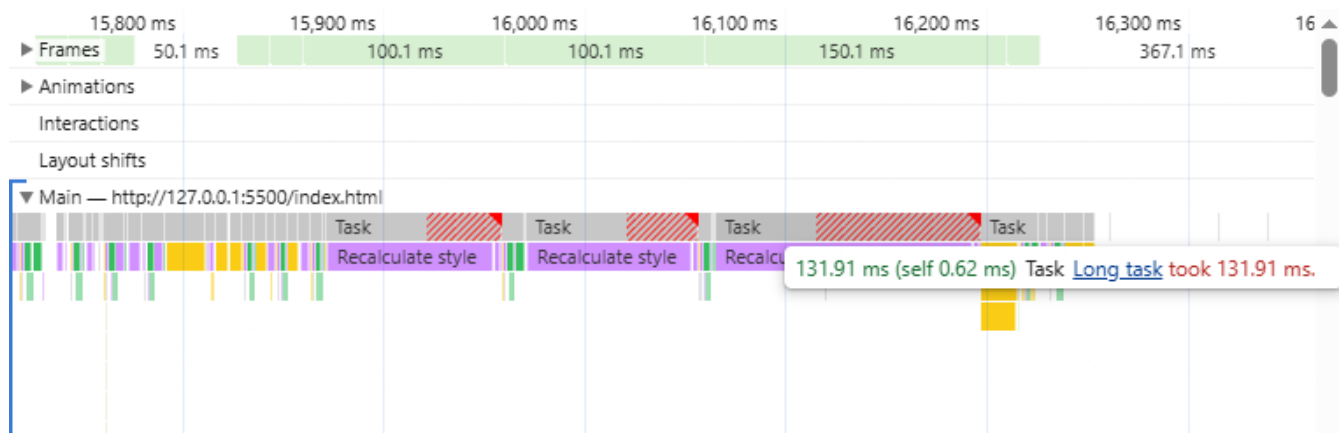


Imagen 4. Captura del Event Log con CPU Throttling x4



Imágen 5. Tiempo de recalculado de estilos con CPU Throttling x4

Podemos ver que la mayoría de tiempo ocurre en la recuperación de los perfiles a través de las llamadas a funciones y procedimientos de JavaScript, mientras que el tiempo de recalculado, si bien largo, no es un tiempo tan extremo, durante el estudio se vio que el selector hover fue el que tenía un % of slow de 100, sin embargo, las optimizaciones para will-change ya las habíamos implementado.